🚀 MTD 367 iOS Application Development

- ~~Lesson #1 - Building Apps That Matter and Swift~~
- **Lesson #2 - Dealing With Swift Types And SwiftUI (Today)**
- Lesson #3 - Doubling Down On SwiftUI (18/8/2025)
- Lesson #4 - All To-Do With SwiftData (1/9/2025)
- Lesson #5 - Streaming Multimedia Swiftly (8/9/2025)
- Lesson #6 - Putting The Map On Your App (15/9/2025)

❓ Questions About Lesson #1

- Course objectives & expectations
- What Swift & SwiftUI are?
- Why they matter?
- Basic Swift language features
  - Variables: `let`, `var`
  - Data types: `String`, `Int`, `Double`, `Bool`, `Optional`
  - Data structure: `Array`
  - Control Statments: `if/else`, `for loop`, `guard`

❓ Questions About Virtual Lab #1

Blackjack doesn’t stop after the first two cards.

- Players can:
  - **Hit** → take another card
  - **Stand** → keep their current total
- Simulate choices with random decisions.
- The next hand starts once a winner has been determined for a single hand.

✅ Today’s Agenda

- □ More Swift Language Features
- □ SwiftUI Basics

🧩 More Swift Language Features

Swift Concepts	Details
🗂️ Data structures	`Dictionary`, `Set`
🧱 Data types	`class`, `struct`, `enum`
📜 Protocols	`CaseIterable`
🔀 Control Statements	`switch`

➡️ Value Vs Reference Types

- Value Types → each copy is separate
- Reference Types → multiple variables point to same object

➡️ Swift Value & Reference Types

Value Types		Reference Types
`Int`	`Set`	`class`
`Double`	`struct`	`Closure`
`Bool`	`enum`	
`String`	`Optional`	
`Array`	`Tuple`	
`Dictionary`		

🚀 Xcode

We will be looking at the above language features and work on Hands-On #3.

🍎✨ SwiftUI Basics

🤦 Ok... Are We Building Apps for the Console?

Users expect buttons, scrolling, navigation, animations and not just text in a terminal!

🍎 Enter SwiftUI

SwiftUI helps you build real interfaces for iOS, iPadOS, macOS & more

- It’s modern, declarative, and backed by Apple
- You’ll learn to build:
- Apps with views
- Respond to user input
- Handle state and data
- Animate beautifully

What Is A Framework?

A toolkit of code written by someone else

- Helps you build apps faster
- Saves you from writing everything from scratch

```
import Foundation // basic tools like dates, strings, and numbers
import UIKit // older framework for building iOS apps
import SwiftUI
```

🎨📝 What Is Declaritive UI?

Instead of telling Swift **how** to create views step by step, you simply **declare** what you want:

- The text
- The color
- The size

SwiftUI takes care of the rest.

🖼️ Views

A **View** is anything you see on the screen

- `Text` → shows words
- `Image` → shows a picture
- `Button` → lets the user tap
- `VStack`, `HStack` → stacks views
- `.font(.title2)`, `.background(.blue)` → modifies views

🔄 What Is `@State`?

`@State` lets a View:

- **Remember values** while your app runs
- Update the screen automatically when the value changes
- Hold simple, local data like numbers, text, or flags

⚡ What Does It Mean When SwiftUI Updates The View?

When a `@State` variable changes:

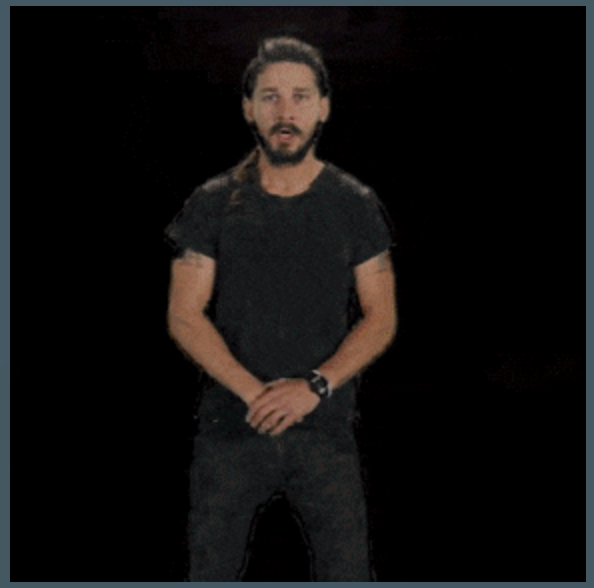
- SwiftUI throws away the old view
- SwiftUI creates a new view reflecting the new data
- You don’t have to write code to update labels or UI elements

🖼️ SwiftUI Vs Storyboards

Feature	Storyboards (2011)	SwiftUI (2019)
UI Creation	Visual drag-and-drop	Code-driven
Framework	UIKit	SwiftUI
Syntax	Imperative	Declarative
Usage	Widely used	Future of Apple development.

🚀 Xcode Again

We will be looking at the above SwiftUI features.



🃏 Hands-On #4

Blackjack UI

- Show the player’s cards and hand total
- Add buttons:
  - **Hit** → draw another card
  - **Stand** → end turn
- Display game messages:
  - 🎉 **Blackjack!**
  - 💥 **Bust!**

✅ Summary

- ✅ More Swift Language Features
  - Data: `enum`, `Struct`, `Class`, `Closure`, `Set`, `Dictionary`
    - Protocol: `CaseIterable`, Control Statments: `switch`
  - Value Types vs Reference Types
- ✅ SwiftUI Basics
  - What Is `@State` & How SwiftUI Updates The View?
  - Views: `Text`, `Button`, `ForEach`, `HStack`, `VStack`
    - Modifiers: `.font(.title2)`, `.background(.blue)`, `.padding()`

🃏 Virtual Lab #2 (16/8/2025)

Keep improving your Blackjack UI!

- There are scenarios where Ace is counted as value 1
- 5-Card Charlie Rule
- Dealer mechanics

❓ Questions?

I’d love to hear how the class went or how I can improve.

Feel free to email me at:
muhammadfadzuli001@suss.edu.sg

📂 All course materials are available at:
www.github.com/iluzdaf/mtd367

📎 Appendix

🗂️ `Dictionary`

- Stores **key-value pairs**
- Keys must be unique

```
let capitals = [
    "Singapore": "Singapore City",
    "Japan": "Tokyo"
]
print(capitals["Japan"]!) // Tokyo
```

Set

- Stores **unique values** (no duplicates)
- Order is **not guaranteed**

```
let colors: Set = ["Red", "Green", "Blue"]
print(colors.contains("Green")) // true
```

`enum`

- define a list of related values in a type-safe way
- Helps you avoid magic strings and makes code safer

```
enum Suit: String {
    case hearts = "♥"
    case diamonds = "♦"
    case clubs = "♣"
    case spades = "♠"
}

let suit = Suit.hearts
```

`struct`

- Define your own data type
- Value type

```
struct Card {
    var rank: String
    var value: Int
    var suit: Suit
}

let card = Card(rank: "Ace", value: 11, suit: .spades)
```

`class`

- Another way to define your own data type
- Reference type

```
class Deck {
    var cards: [Card] = []

    func shuffle() {
        cards.shuffle()
    }

    func dealCard() -> Card? {
        return cards.popLast()
    }
}
```

Why choose to use class for Deck?

- Deck represents a single shared object in the game.
- Players all draw cards from the SAME deck.
- If one card is dealt, it's removed from the deck for everyone.
- Classes are REFERENCE TYPES:
  - All variables pointing to the deck share the same instance.
  - Changes made by one player affect the shared deck.
- A struct would create a COPY each time it's passed around.
  - Each player might accidentally get their own deck, which breaks the game logic.

```
class Deck {
    var cards: [String] = ["Ace", "King", "Queen"]

    func dealCard() -> String? {
        return cards.popLast()
    }
}

let originalDeck = Deck()
let copiedDeck = originalDeck

copiedDeck.dealCard()

originalDeck.cards // ["Ace", "King"]
copiedDeck.cards // ["Ace", "King"]
```

```swift
struct DeckStruct {
    var cards: [String] = ["Ace", "King", "Queen"]

    mutating func dealCard() -> String? {
        return cards.popLast()
    }
}

var originalDeckStruct = DeckStruct()
var copiedDeckStruct = originalDeckStruct

copiedDeckStruct.dealCard()

originalDeckStruct.cards // ["Ace", "King", "Queen"]
copiedDeckStruct.cards // ["Ace", "King"]
```

One Button Adventure

```
import SwiftUI
import PlaygroundSupport

struct ContentView: View {
    @State private var score = 0
    let suits = ["♥", "♦", "♣", "♠"]

    var body: some View {
        VStack(spacing: 20) {
            Text("Score: \(score)")
                .font(.largeTitle)
                .frame(width: 300)

            Button("Add 1") {
                score += 1
            }
            .padding()
            .background(Color.blue)
            .foregroundColor(.white)
            .cornerRadius(10)

            HStack {
                ForEach(suits, id: \.self) { suit in
                    Text(suit)
                        .font(.largeTitle)
                        .padding()
                }
            }
        }
        .padding()
    }
}

PlaygroundPage.current.setLiveView(ContentView())
```